PRACTICAL - 1

Implementation and Time Analysis of Sorting Algorithms

Aim of the Practical:

To implement Bubble Sort, Selection Sort, Insertion Sort, Merge Sort and Quick Sort in C++ and analyze their time and space complexities.

1.1 Bubble Sort

Aim:

To implement optimized Bubble Sort and derive its best, average and worst-case time complexity.

Program:

```
#include <iostream>
using namespace std;
void bubbleSort(int a[], int n) {
    for (int i = 0; i < n - 1; i++) {
        bool swapped = false;
        for (int j = 0; j < n - i - 1; j++) {
            if (a[j] > a[j + 1]) {
                swap(a[j], a[j + 1]);
                swapped = true;
            }
        }
        if (!swapped) break;
    }
}
int main() {
    int a[] = {64, 34, 25, 12, 22, 11, 90, 45, 78, 3};
    int n = sizeof(a) / sizeof(a[0]);
    cout << "Original array: ";
    for (int x : a) cout << x << " ";
    bubbleSort(a, n);
    cout << "\nSorted array: ";
    for (int x : a) cout << x << " ";
}
```

Output:

```
Original array: 64 34 25 12 22 11 90 45 78 3
Sorted array: 3 11 12 22 25 34 45 64 78 90
```

Time Analysis:

Derivation of Time Complexity:

Let n be the number of elements in the array.

Outer loop runs at most (n - 1) times.

For each outer-loop value i, the inner loop runs (n - i - 1) times.

$$C(n) = \Sigma(i = 0 \text{ to } n - 2)\ (n - i - 1)$$

Re-indexing with k = n - i - 1 gives:

$$C(n) = \Sigma(k = 1 \text{ to } n - 1)\ k$$

$$C(n) = 1 + 2 + 3 + \ldots + (n - 1)$$

$$C(n) = n(n - 1) / 2 = (n^2 - n) / 2$$

Each comparison and possible swap takes constant time, so:

$$T(n) = c1 \cdot C(n) + c2$$

$$T(n) = (c1/2)n^2 - (c1/2)n + c2$$

Dropping constants and lower-order terms gives $T(n) = \Theta(n^2)$ for average/worst cases.

Name: DAYANAND MARNENI Enrollment No: 92400118442

In the optimized version, if the array is already sorted, swapped remains false after the first pass. The inner loop then performs only (n - 1) comparisons.

$$T_{best}(n) = c(n - 1) + c0 = \Theta(n)$$

Final Time Analysis:

- Best Case: $\Theta(n)$ - already sorted array with the swapped flag.
- Average Case: $\Theta(n^2)$.
- Worst Case: $\Theta(n^2)$ - reverse-sorted array.
- Auxiliary Space: $\Theta(1)$.

Result:

Optimized Bubble Sort was implemented successfully. The derived complexity matches the program structure.

1.2 Selection Sort

Aim:

To implement Selection Sort and derive its time complexity from the number of comparisons.

Program:

```
#include <iostream>
using namespace std;
void selectionSort(int a[], int n) {
    for (int i = 0; i < n - 1; i++) {
        int minIndex = i;
        for (int j = i + 1; j < n; j++) {
            if (a[j] < a[minIndex]) minIndex = j;
        }
        swap(a[i], a[minIndex]);
    }
}
int main() {
    int a[] = {64, 34, 25, 12, 22, 11, 90, 45, 78, 3};
    int n = sizeof(a) / sizeof(a[0]);
    selectionSort(a, n);
    for (int x : a) cout << x << " ";
}
```

Output:

```
3 11 12 22 25 34 45 64 78 90
```

Time Analysis:

Derivation of Time Complexity:

Let n be the number of elements.

The outer loop runs (n - 1) times.

For i = 0, the inner loop performs (n - 1) comparisons; for i = 1 it performs (n - 2), and so on.

$$C(n) = (n - 1) + (n - 2) + \ldots + 1$$

$$C(n) = \Sigma(k = 1 \text{ to } n - 1)\ k$$

$$C(n) = n(n - 1) / 2$$

Unlike Bubble or Insertion Sort, the number of comparisons does not depend on the initial order.

The algorithm performs at most one swap per outer iteration:

$$S(n) = n - 1$$

Hence:

$$T(n) = c1 \cdot n(n - 1)/2 + c2 \cdot (n - 1) + c3$$

$$T(n) = \Theta(n^2)$$

Final Time Analysis:

- Best Case: $\Theta(n^2)$.
- Average Case: $\Theta(n^2)$.
- Worst Case: $\Theta(n^2)$.
- Auxiliary Space: $\Theta(1)$.

Result:

Selection Sort was implemented successfully and its quadratic time complexity was derived from the fixed number of comparisons.

1.3 Insertion Sort

Aim:

To implement Insertion Sort and derive its best and worst-case complexities.

Program:

```
#include <iostream>
using namespace std;
void insertionSort(int a[], int n) {
    for (int i = 1; i < n; i++) {
        int key = a[i];
        int j = i - 1;
        while (j >= 0 && a[j] > key) {
            a[j + 1] = a[j];
            j--;
        }
        a[j + 1] = key;
    }
}
int main() {
    int a[] = {64, 34, 25, 12, 22, 11, 90, 45, 78, 3};
    int n = sizeof(a) / sizeof(a[0]);
    insertionSort(a, n);
    for (int x : a) cout << x << " ";
}
```

Output:

```
3 11 12 22 25 34 45 64 78 90
```

Time Analysis:

Derivation of Time Complexity:

Let n be the number of elements.

The outer loop runs from i = 1 to n - 1, so it runs (n - 1) times.

Worst case: the array is in reverse order. For each i, the key must be shifted across all i previous elements.

$$C_{worst}(n) = \Sigma(i = 1 \text{ to } n - 1)\ i$$
$$C_{worst}(n) = 1 + 2 + \ldots + (n - 1)$$
$$C_{worst}(n) = n(n - 1)/2$$
$$T_{worst}(n) = \Theta(n^2)$$

Best case: the array is already sorted. The while-condition fails after the first comparison in every outer-loop iteration.

$$C_{best}(n) = \Sigma(i = 1 \text{ to } n - 1)\ 1 = n - 1$$
$$T_{best}(n) = \Theta(n)$$

For random input, approximately half of the already-sorted portion is shifted on average, which still gives $\Theta(n^2)$.

Final Time Analysis:

- Best Case: Θ(n).
- Average Case: Θ(n^2).
- Worst Case: Θ(n^2).
- Auxiliary Space: Θ(1).

Result:

Insertion Sort was implemented successfully. The step-by-step count explains why nearly sorted data is favorable.

1.4 Merge Sort

Aim:

To implement Merge Sort using divide and conquer and solve its recurrence using the Master Method.

Program:

```
#include <iostream>
#include <vector>
using namespace std;
void merge(int a[], int l, int m, int r) {
    vector<int> temp;
    int i = l, j = m + 1;
    while (i <= m && j <= r)
        temp.push_back(a[i] <= a[j] ? a[i++] : a[j++]);
    while (i <= m) temp.push_back(a[i++]);
    while (j <= r) temp.push_back(a[j++]);
    for (int k = 0; k < (int)temp.size(); k++)
        a[l + k] = temp[k];
}
void mergeSort(int a[], int l, int r) {
    if (l >= r) return;
    int m = l + (r - l) / 2;
    mergeSort(a, l, m);
    mergeSort(a, m + 1, r);
    merge(a, l, m, r);
}
int main() {
    int a[] = {64, 34, 25, 12, 22, 11, 90, 45, 78, 3};
    int n = sizeof(a) / sizeof(a[0]);
    mergeSort(a, 0, n - 1);
    for (int x : a) cout << x << " ";
}
```

Output:

```
3 11 12 22 25 34 45 64 78 90
```

Time Analysis:

Time Complexity using Master Method:

Merge Sort divides the input into two equal halves and merges the two sorted halves in linear time.

$$T(n) = 2T(n/2) + cn$$

Compare with the standard Master Theorem form:

$$T(n) = aT(n/b) + f(n)$$

$$a = 2, \quad b = 2, \quad f(n) = cn = \Theta(n)$$

Now compute:

$$n^{(\log_b a)} = n^{(\log_2 2)} = n^1 = n$$

Since:

$$f(n) = \Theta(n) = \Theta(n^{(\log_b a)})$$

this is Master Theorem Case 2.

$T(n) = \Theta(n^{(\log_b a)} \cdot \log n)$

$T(n) = \Theta(n \log n)$

The input is always divided and merged in the same manner, so best, average and worst cases are all Θ(n log n).

Final Time Analysis:

- Best Case: Θ(n log n).
- Average Case: Θ(n log n).
- Worst Case: Θ(n log n).
- Auxiliary Space: Θ(n).

Result:

Merge Sort was implemented successfully and its recurrence was solved step by step using the Master Method.

1.5 Quick Sort

Aim:

To implement Quick Sort and use the Master Method for the balanced recurrence while deriving the worst case separately.

Program:

```
#include <iostream>
using namespace std;
int partitionArray(int a[], int low, int high) {
    int pivot = a[high];
    int i = low - 1;
    for (int j = low; j < high; j++) {
        if (a[j] <= pivot) {
            i++;
            swap(a[i], a[j]);
        }
    }
    swap(a[i + 1], a[high]);
    return i + 1;
}
void quickSort(int a[], int low, int high) {
    if (low < high) {
        int p = partitionArray(a, low, high);
        quickSort(a, low, p - 1);
        quickSort(a, p + 1, high);
    }
}
int main() {
    int a[] = {64, 34, 25, 12, 22, 11, 90, 45, 78, 3};
    int n = sizeof(a) / sizeof(a[0]);
    quickSort(a, 0, n - 1);
    for (int x : a) cout << x << " ";
}
```

Output:

```
3 11 12 22 25 34 45 64 78 90
```

Time Analysis:

Time Complexity using Master Method:

Partitioning scans the current subarray once, so partition cost is Θ(n).

Best/balanced case: the pivot divides the array into two approximately equal halves.

$T(n) = 2T(n/2) + cn$

Using Master Theorem:

$a = 2, \quad b = 2, \quad f(n) = \Theta(n)$

$n^{(\log_b a)} = n^{(\log_2 2)} = n$

$f(n) = \Theta(n^{(\log_b a)})$

Hence Master Theorem Case 2 applies:

$T_{best}(n) = \Theta(n \log n)$

Average case also produces $\Theta(n \log n)$ expected running time.

Worst case: if the pivot is repeatedly the smallest or largest element, one side has n - 1 elements and the other side has 0.

$T(n) = T(n - 1) + cn$

This recurrence is not in aT(n/b)+f(n) form, so the ordinary Master Theorem does not apply.

Unrolling:

$$\begin{aligned} T(n) &= T(n - 1) + cn \\ &= T(n - 2) + c(n - 1) + cn \\ &= T(1) + c(2 + 3 + \ldots + n) \\ &= \Theta(n^2) \end{aligned}$$

Final Time Analysis:

- Best Case: $\Theta(n \log n)$.
- Average Case: $\Theta(n \log n)$.
- Worst Case: $\Theta(n^2)$.
- Average recursion stack: $\Theta(\log n)$.
- Worst-case recursion stack: $\Theta(n)$.

Result:

Quick Sort was implemented successfully. The balanced case was solved using the Master Method, and the unbalanced worst case was derived separately.

Conclusion:

All five sorting algorithms were implemented and their time complexities were derived step by step. Merge Sort and balanced/average Quick Sort achieve $\Theta(n \log n)$, while Bubble, Selection and Insertion Sort are quadratic in their general or worst cases.